EWB CLOUD LABS / ENGINEERING PROJECT

AWS VPC Peering

Two private workloads.
One direct VPC connection.

Build, test and explain private connectivity using EC2, custom route tables, security groups and a bastion host.

Submission detail	Value
Student	Manasa
Region	Europe (Frankfurt) / eu-central-1
Execution dates	18 October 2026 (Mac); 19 October 2026 (Windows)
Provisioning	AWS CloudFormation / ewb-vpc-peering-lab
Validation	Baseline, two-way ping, private SSH, failure/recovery; Windows repeat tests
Report status	Technical validation complete; cleanup pending after demonstration

Prepared from the student-provided AWS Console screenshots and terminal evidence. Batch information was not supplied.

Retention note: The setup is being retained for 6-7 days for a possible demonstration. Instance stopping and final cleanup have not yet been evidenced.

PROJECT REPORT

Reading guide and key terms

This project proves that two servers without public IPs can communicate through VPC peering. A third server, the bastion, provides the secure SSH entry path. Read the design first, then use the labelled screenshots to verify each claim.

Where to look	Pages
Architecture, resource IDs, routing and security	3-5
Implementation, observed tests and lessons	6-7
Retention and pending cleanup	8
Windows results, commands and full SSH config	9-11
Original Mac tests and AWS configuration evidence	12-30
Windows screenshots / W01-W02	31-32
Sources and final completion status	33

Key terms in plain language

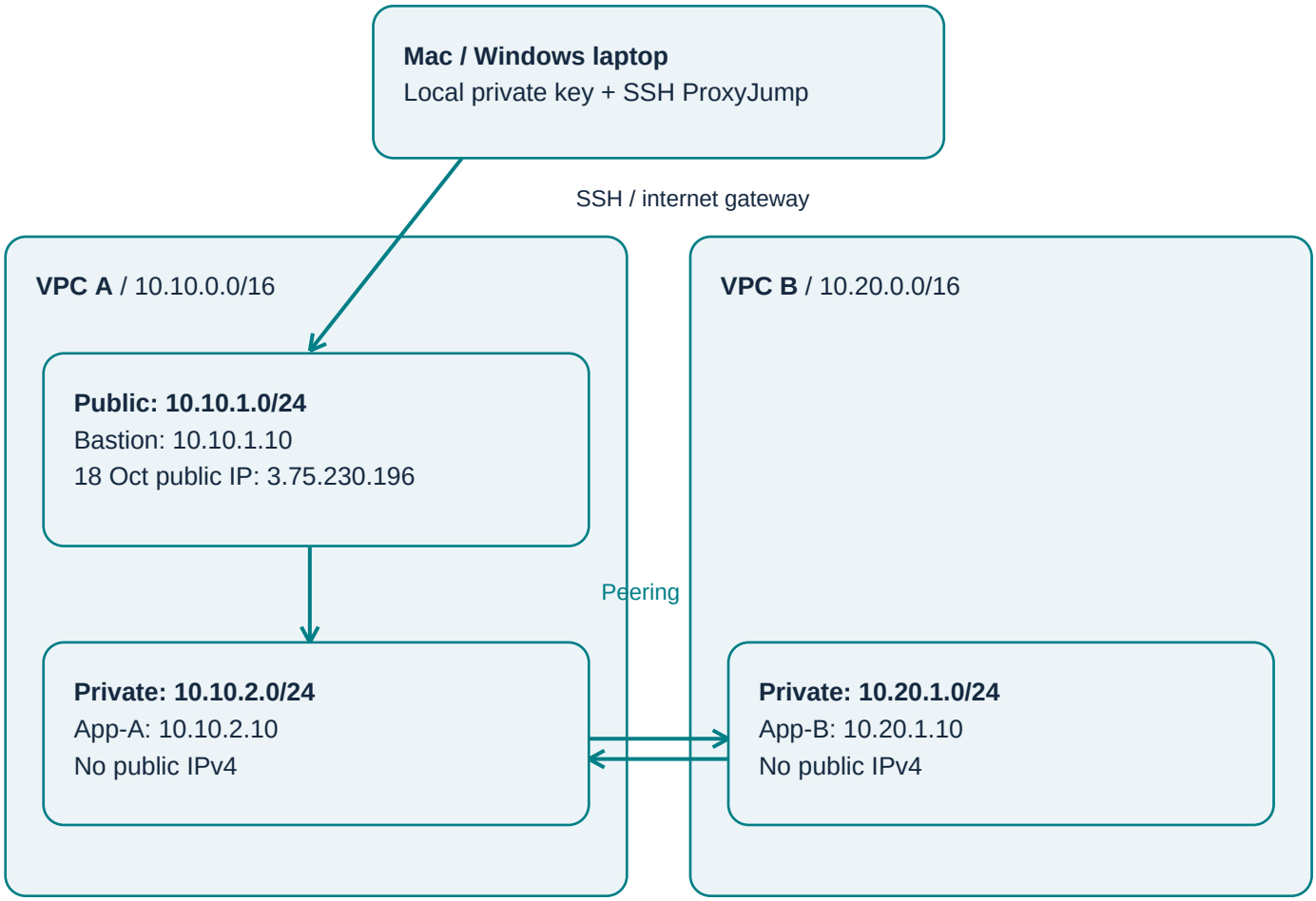
Term	Meaning in this project
VPC / subnet	A private AWS network / a smaller address range within that network.
CIDR /32	CIDR writes an address range. A /32 IPv4 source allows one specific address.
Route table	Chooses the next destination for network traffic. A route does not grant firewall permission.
Security group	Instance firewall rules that permit the required traffic.
Bastion / ProxyJump	Public administration host / SSH option that connects through that host.
SSH / ICMP	SSH opens a remote terminal. Ping uses ICMP Echo Request and Echo Reply to test reachability.
Internet gateway / peering	Internet entry for VPC A / private connection between VPC A and VPC B.
CloudFormation stack	The collection of AWS resources created and managed from the template.

Evidence labels E01-E16b refer to the original build and tests on 18 October. W01-W02 show Windows repeat tests on 19 October. Cleanup remains pending until the live demonstration is finished.

Architecture and traffic paths

Design objective

Connect two non-overlapping VPCs in one AWS account and Region. Both application servers use private IPv4 addresses only. A public bastion is the administration entry point; no NAT gateway is required.



Administration: Mac or Windows laptop to bastion over public SSH; bastion opens the connection to each private host. The private key remains on the local laptop.

Project traffic: App-A to App-B follows VPC A's private route table and peering connection. Replies use VPC B's route back to 10.10.0.0/16. The reverse ping validates initiation from App-B as well.

Isolation: VPC B has no internet gateway. Neither private subnet has a default internet route. Peering does not provide internet transit through VPC A.

Actual resource inventory

Resource	AWS ID / captured address
VPC A	vpc-0052adfdbaea7de1b
VPC B	vpc-044f1d87e0ac4688d
Public subnet A	subnet-08b29fb274ca7625a / 10.10.1.0/24
Private subnet A	subnet-07cbab12f7c9a61b6 / 10.10.2.0/24
Private subnet B	subnet-0056cc81b779e6456 / 10.20.1.0/24
Bastion	i-0e35bcb896efbe473 / 10.10.1.10
App-A	i-08bc2fd22586f60bd / 10.10.2.10
App-B	i-0ba9d27bd7ca58329 / 10.20.1.10
Internet gateway	igw-0373b2dcd368e6911
Peering connection	pcx-0e32f06d76daeeba3
Public route table A	rtb-01085589da9210ca5
Private route table A	rtb-0181641a7e4ff8260
Private route table B	rtb-000680f873aa0936e
Bastion security group	sg-0626d02680dd28298
App-A security group	sg-075f1105cd876a8d7
App-B security group	sg-0d3de18dc49f34271

Captured compute: Amazon Linux 2023, t3.micro, eu-central-1a. The bastion public address may change after stop/start. IDs are transcribed from the supplied Outputs and EC2 screenshots.

PROJECT REPORT

Routing and security design

Route tables and explicit subnet associations

Table / subnet	Destination	Target
A public / ewb-a-public	10.10.0.0/16 0.0.0.0/0 10.20.0.0/16	local internet gateway peering
A private / ewb-a-private	10.10.0.0/16 10.20.0.0/16	local peering
B private / ewb-b-private	10.20.0.0/16 10.10.0.0/16	local peering

The bastion subnet also needs the peer route because ProxyJump opens the App-B SSH connection from the bastion. The specific peer route is preferred over the public default route.

Inbound security rules and dated SSH sources

Group	Traffic	Allowed source
ewb-sg-bastion	SSH / TCP 22	18 Oct: 106.200.31.238/32 19 Oct: 106.200.28.37/32
ewb-sg-app-a	SSH / TCP 22 ICMP Echo Request	10.10.1.10/32 10.20.1.10/32
ewb-sg-app-b	SSH / TCP 22 ICMP Echo Request	10.10.1.10/32 10.10.2.10/32

ICMP Echo Request uses type 8, code 0. Stateful security groups allow replies to permitted traffic, but return routes are still needed. The template keeps outbound allow-all and default network ACLs. The dated bastion SSH sources are successive settings, not two simultaneous rules.

Implementation and test results

Provisioning used the CloudFormation template rather than manual creation of every resource. EnablePeering=false produced the baseline. Updating it to true created the connection and three peer routes. Same-account peering acceptance was automated by CloudFormation.

Test	Observed result	Evidence
Baseline: App-A to 10.20.1.10	4 sent, 0 received; 100% loss	E01
Forward: App-A to App-B	4 sent, 4 received; 0% loss	E02
Reverse: App-B to App-A	4 sent, 4 received; 0% loss	E03
Private SSH to App-B	Shell on 10.20.1.10 via bastion	E03
Controlled failure	4 sent, 0 received; 100% loss	E04
Recovery	4 sent, 4 received; 0% loss	E05
Private host isolation	No public IPv4 on App-A or App-B	E14-E15

Controlled change

With peering still enabled, AllowForwardPing was set to false through a stack update. This removed only App-B's inbound Echo Request rule from 10.10.2.10/32. SSH remained available. After the failure was captured, AllowForwardPing=true restored the rule and successful ping.

Repeatable terminal validation

From Mac Terminal or Windows PowerShell: `ssh ewb-app-a` or `ssh ewb-app-b`.
Inside the chosen private host: `hostname -I`, then `ping -c 4 -W 2 PEER_PRIVATE_IP`.
The ewb-* aliases are defined in the local laptop SSH config, not inside EC2. Replace PEER_PRIVATE_IP with the other private server IP.

Troubleshooting and learning

Regional account validation

Initial Frankfurt and Ohio launches were blocked by AWS account validation. After the student reported Frankfurt validation complete, the rolled-back stack was deleted and recreated successfully in Frankfurt.

SSH source IP changed

The laptop public IPv4 changed from 106.200.24.163 to 106.200.31.238. Updating LaptopPublicIPv4 to the new /32 restored reachability without opening SSH to the internet.

SSH session interruption

An initial connection closed, and a ProxyJump attempt reported a broken pipe. A later attempt with SSH keepalive options connected successfully. The logs do not establish the underlying cause of the interruptions.

Local terminal versus remote shell

In this lab, ping run directly from Mac Terminal or Windows PowerShell cannot reach these private addresses; first connect by SSH to the private host. The valid tests show the App-A or App-B prompt and private host IP before ping. An SSH alias entered inside EC2 does not resolve because the alias is defined on the local laptop.

Viva essentials

Why return routes? Replies need a route back to the source VPC.
Why is Active not sufficient? Subnet routes and security rules must also permit the traffic.
Why is App-B's SSH source 10.10.1.10? ProxyJump opens the target connection from the bastion.
Does A-B plus A-C connect B-C? No. VPC peering is non-transitive.
Can VPC B use A's internet gateway? No, peering does not provide that transit.

Retention and cleanup status

Current status: cleanup pending

The student requested retaining the lab for 6-7 days for a possible demonstration. This report does not claim the stack has been deleted or that the instances have been stopped. No cleanup evidence has been supplied.

During the retention period

Stop only the three lab instances in EC2 when not in use, and verify their Stopped state. Keep the CloudFormation stack. EBS storage remains chargeable. Before demonstrating, start all three instances and wait for status checks. Read the bastion's current public IPv4 directly from EC2 and update its HostName in the local laptop SSH configuration. Check the current laptop public IPv4 against the SSH source rule.

Cleanup after the demonstration

1. Save all report evidence and confirm a live setup is no longer needed.

2. Delete only the ewb-vpc-peering-lab CloudFormation stack in Frankfurt. This terminates its instances and removes the stack-owned networking and delete-on-termination disks.

3. Verify DELETE_COMPLETE in the deleted-stack view. Capture that state and the three terminated instance IDs. Inspect EC2 Volumes for leftover lab disks.

4. Check the lab VPCs, peering and any public IPv4 allocations. Review earlier attempts in Ohio separately; this Frankfurt stack cannot clean up another Region.

5. The separately created key pair is outside the stack. Remove it and its local private key only when no longer needed. Check Billing afterward; reporting can lag.

Before final submission

Append real cleanup screenshots and update this section with the actual cleanup date. Until then, this is a technical completion report with final cleanup outstanding.

Windows validation / 19 October

The same Frankfurt infrastructure was accessed from Windows PowerShell using OpenSSH_for_Windows 9.5p2. The original Mac baseline, controlled failure and recovery evidence remains unchanged. The two new screenshots demonstrate successful forward and reverse connectivity from Windows SSH sessions.

Windows repeat test	Actual result	Evidence
App-A to App-B	10.10.2.10 to 10.20.1.10; 4/4 replies, 0% loss; average 0.359 ms	W01
App-B to App-A	10.20.1.10 to 10.10.2.10; 4/4 replies, 0% loss; average 0.204 ms	W02

Connection settings used for the repeat tests

Setting	Value recorded on 19 October
Bastion public IPv4	51.102.185.255 after restart
Laptop public IPv4 / SSH source	106.200.28.37/32
Windows user profile	C:\Users\ramyasree
Private key location	C:\Users\ramyasree\Downloads\ewb-lab-new.pem
SSH configuration file	C:\Users\ramyasree\.ssh\config (no .txt extension)

Windows paths use the existing ramyasree user profile. On another Windows account, replace that profile segment with its actual username. Original screenshots are preserved.

Public IP output and stack update recovery

An update reported that the PublicIp attribute did not exist while the bastion was stopped. Starting the instances restored the public address and the student reported that the update worked. For future updates, start the bastion first because the current template output reads its PublicIp attribute. Read its current IP directly from EC2 after every stop/start.

Windows demonstration commands

Run SSH aliases from Windows PowerShell. After SSH connects, hostname and ping run inside the remote Amazon Linux shell. Confirm the prompt before each test.

1 / Check local access and connect

```
ssh -V
curl.exe -4 https://checkip.amazonaws.com
Test-Path "$env:USERPROFILE\Downloads\ewb-lab-new.pem"
ssh ewb-app-a
```

Use the matching private key and a local SSH config with Host entries for ewb-bastion, ewb-app-a and ewb-app-b. Each application host uses ProxyJump ewb-bastion. IdentityFile points to the key on Windows. If Notepad saved config.txt, rename it to config.

2 / App-A shell: forward ping

```
hostname -I
ping -c 4 -W 2 10.20.1.10
exit
```

Expected host IP: 10.10.2.10. Save the successful ping summary.

3 / PowerShell, then App-B shell: reverse ping

```
ssh ewb-app-b
```

After the App-B shell appears:

```
hostname -I
ping -c 4 -W 2 10.10.2.10
exit
```

Expected host IP: 10.20.1.10. Save the successful ping summary.

Before a later demonstration

Start all three instances, wait for status checks, update the bastion HostName in the local config to its current EC2 public IP, and update LaptopPublicIPv4 in the stack to the current Windows public IP /32. Keep EnablePeering=true and AllowForwardPing=true for positive tests. Do not copy the private key to an EC2 instance.

Windows SSH configuration

Save this text in `C:/Users/ramyasree/.ssh/config`, without a .txt extension. It contains connection settings, not the private key itself. The matching .pem stays in Downloads on this Windows laptop.

The bastion address below was recorded on 19 October. After stop/start, replace its HostName with the current public IPv4 shown by EC2. The two application private IPs remain fixed for these existing instances.

```
Host ewb-bastion
    HostName 51.102.185.255
    User ec2-user
    IdentityFile C:/Users/ramyasree/Downloads/ewb-lab-new.pem
    IdentitiesOnly yes
    ServerAliveInterval 15

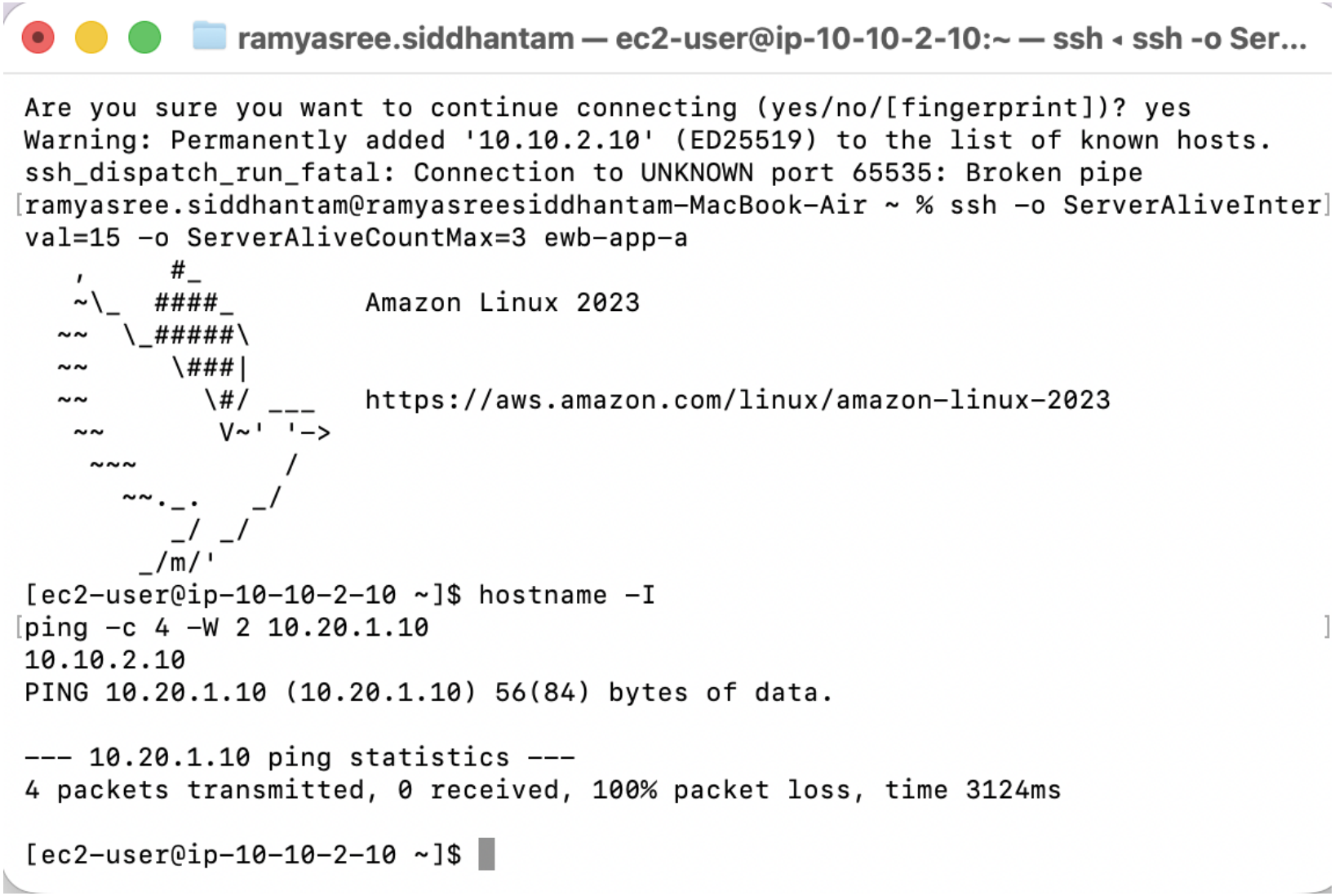
Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile C:/Users/ramyasree/Downloads/ewb-lab-new.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
    ServerAliveInterval 15

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile C:/Users/ramyasree/Downloads/ewb-lab-new.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
    ServerAliveInterval 15
```

On a first connection, compare the server host fingerprint with the trusted instance system log before accepting it. Do not disable host-key checking or copy the private key onto the bastion.

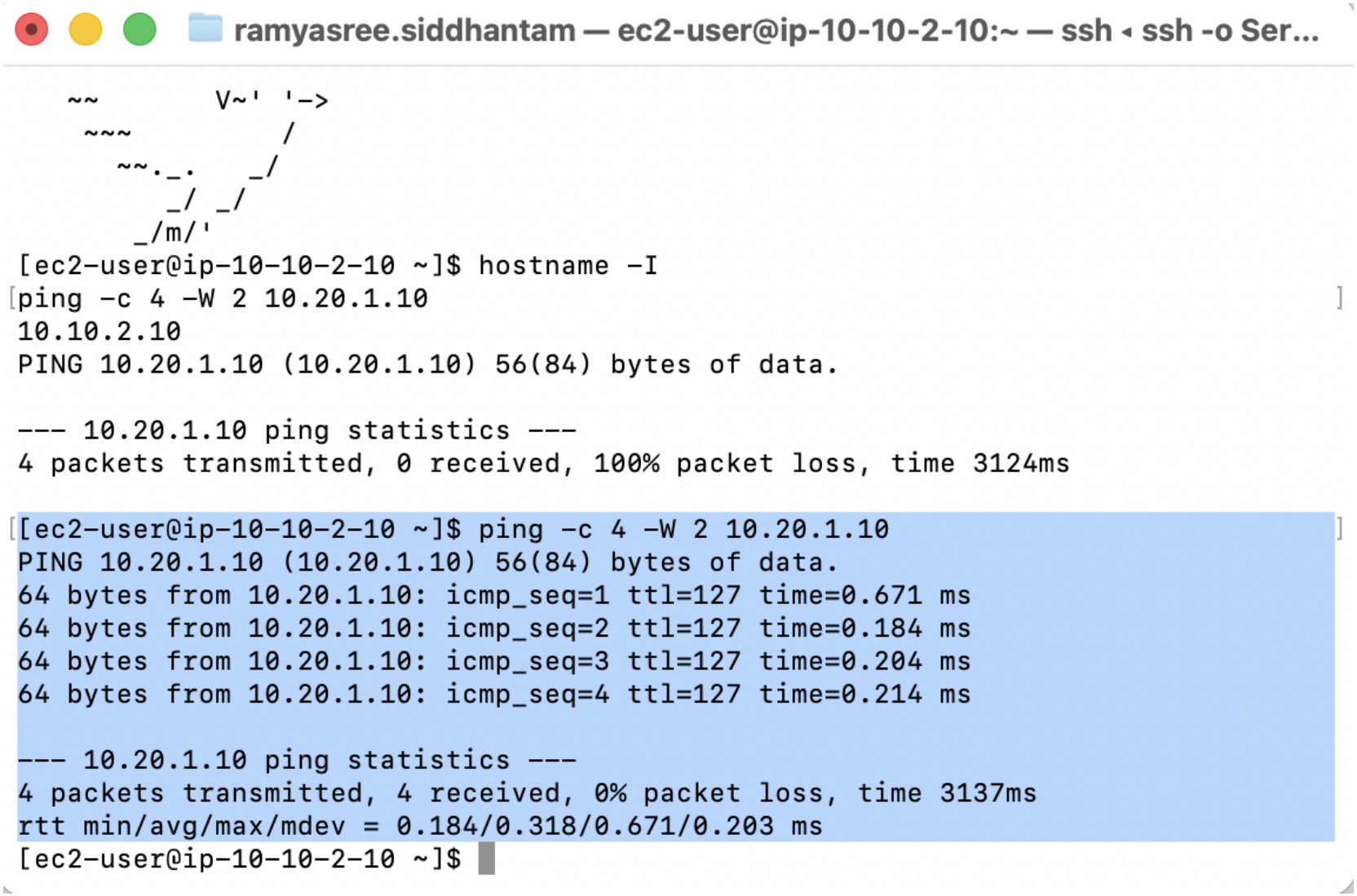
Baseline failure

App-A / 10.10.2.10. Before peering, the App-B ping receives no replies. The screenshot also includes an earlier SSH retry.



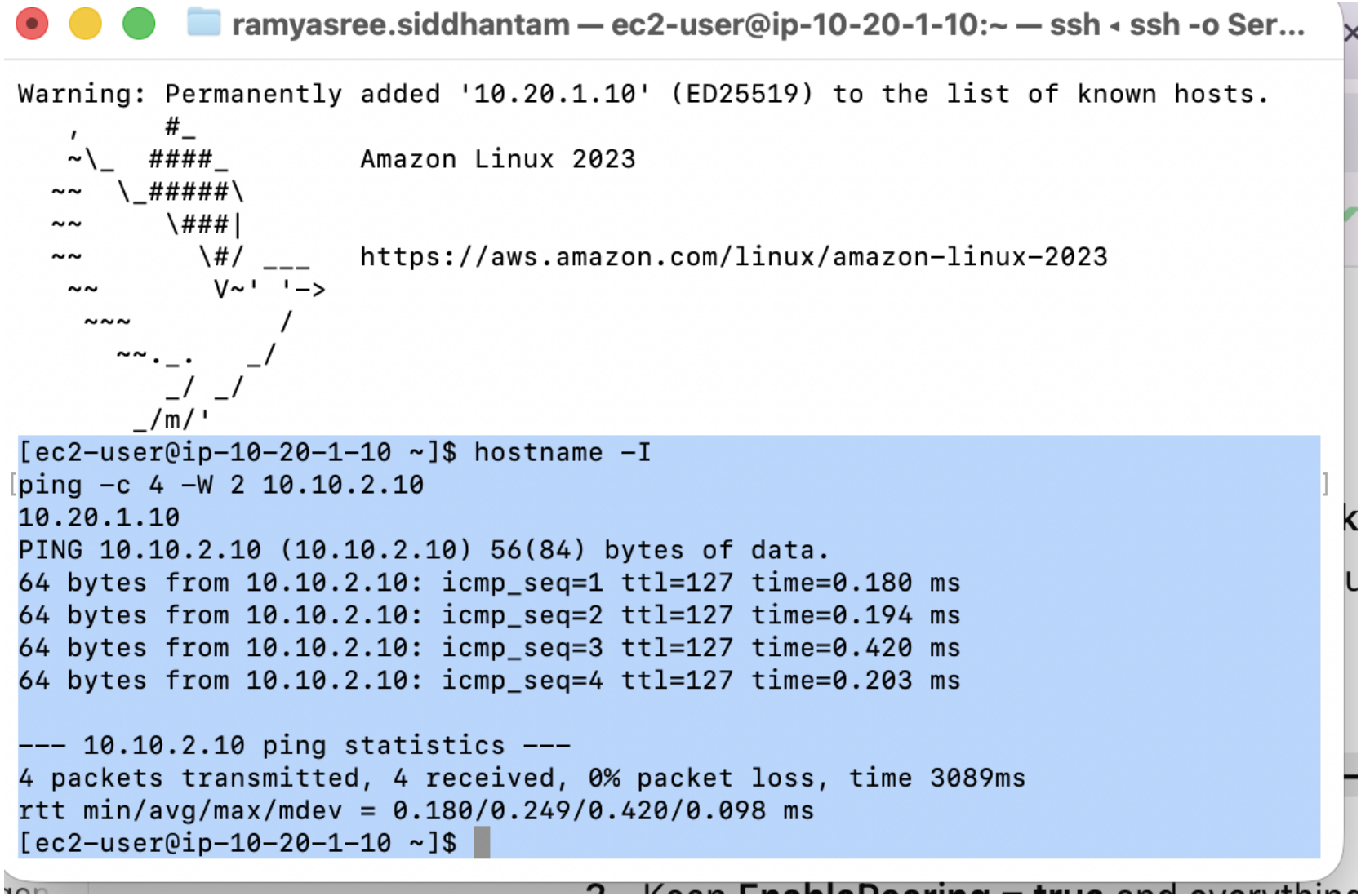
Forward connectivity

App-A to 10.20.1.10: four replies and 0% packet loss after enabling peering. The preceding baseline is visible above.



Private SSH and reverse connectivity

App-B shell and hostname confirm 10.20.1.10. Ping to App-A 10.10.2.10 succeeds with four replies.



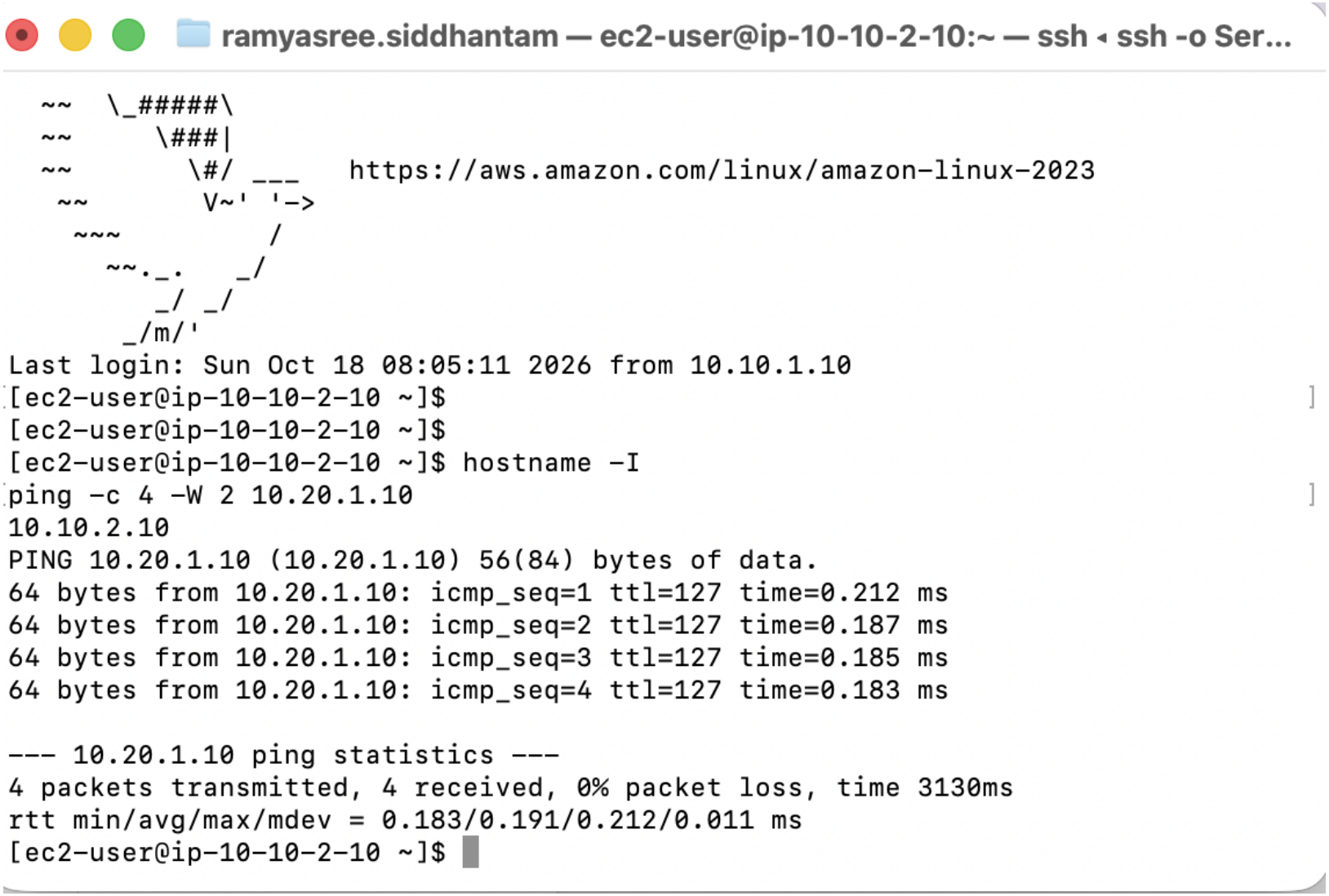
Controlled ICMP failure

The lower test runs on App-A: four packets sent, none received after removal of App-B inbound ICMP. Earlier local-alias errors are unrelated to the final ping.



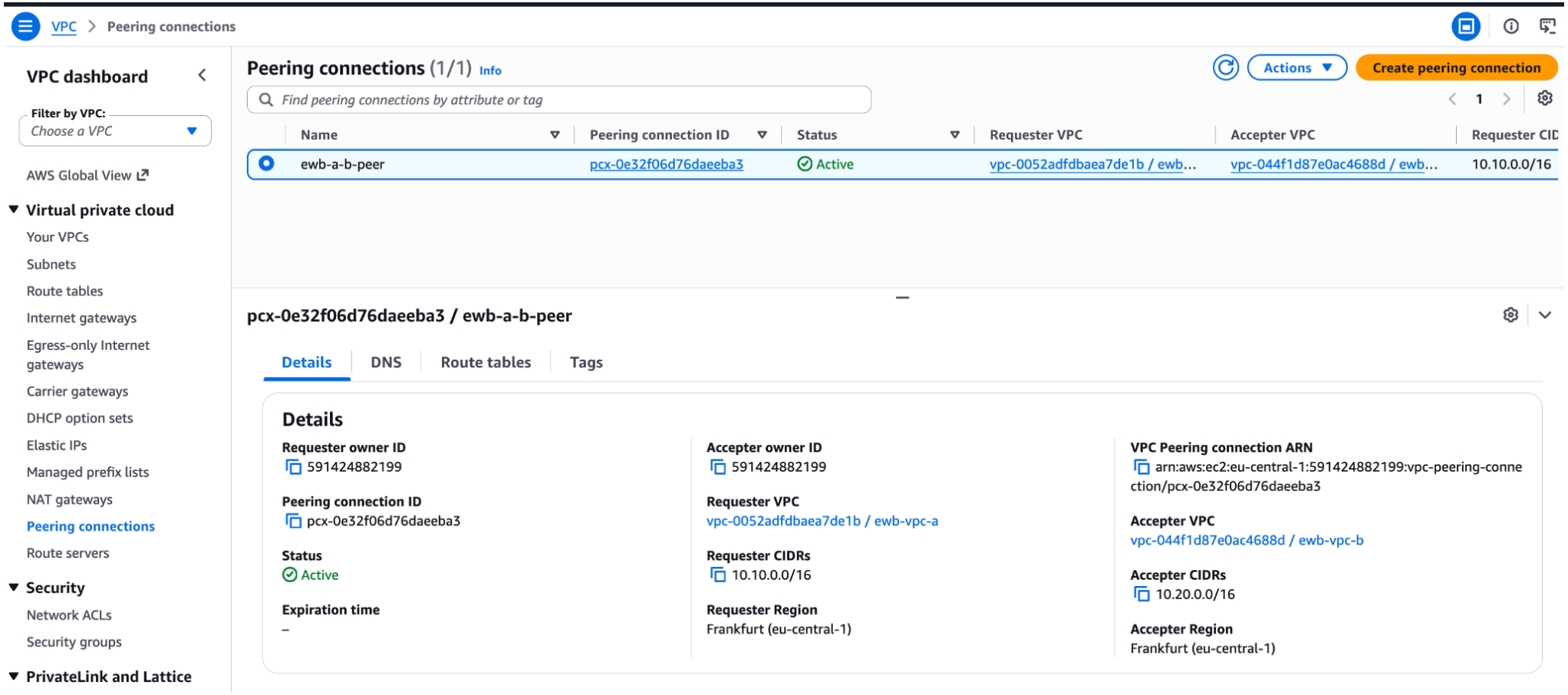
Recovery after restoring the rule

App-A private IP and successful ping to App-B: four replies, 0% loss. Login source 10.10.1.10 is consistent with the bastion path.



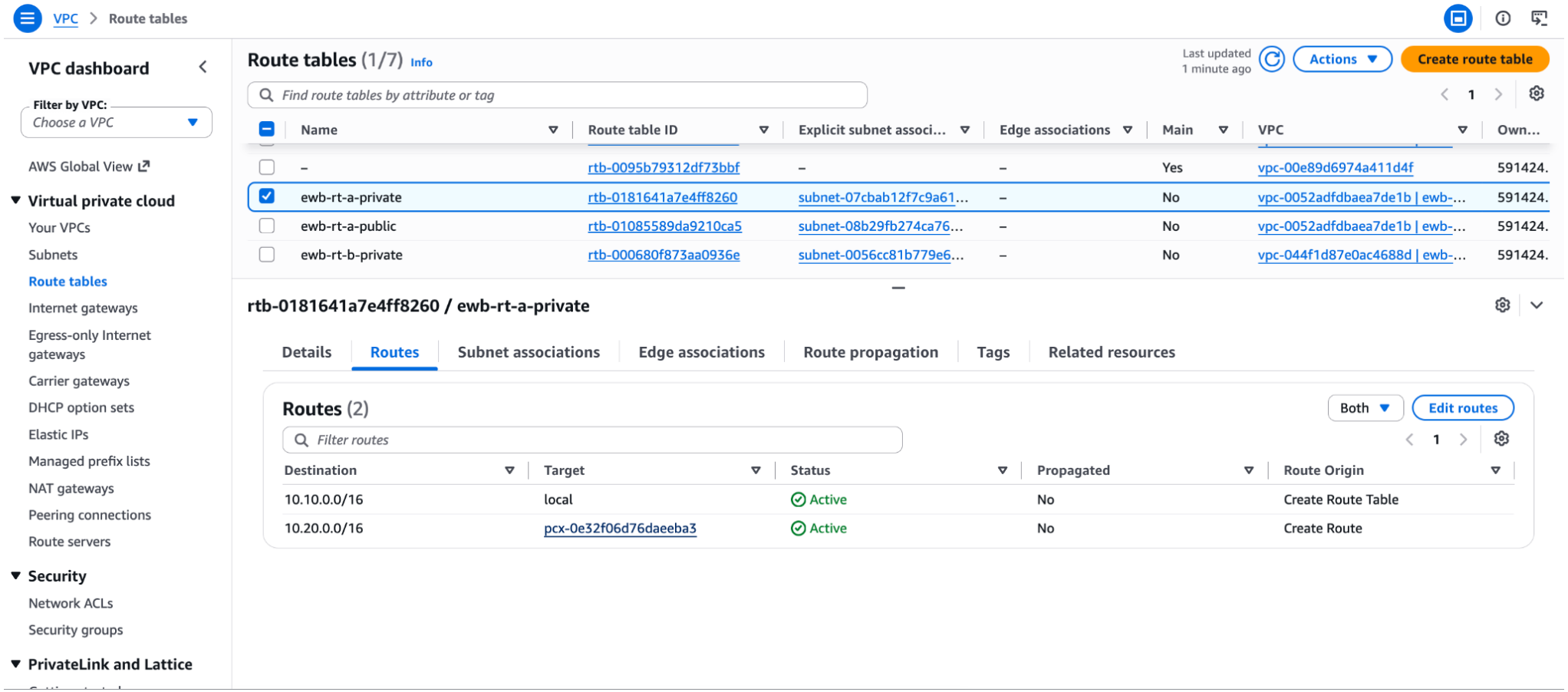
Active peering connection

ewb-a-b-peer is Active between VPC A 10.10.0.0/16 and VPC B 10.20.0.0/16 in Frankfurt.



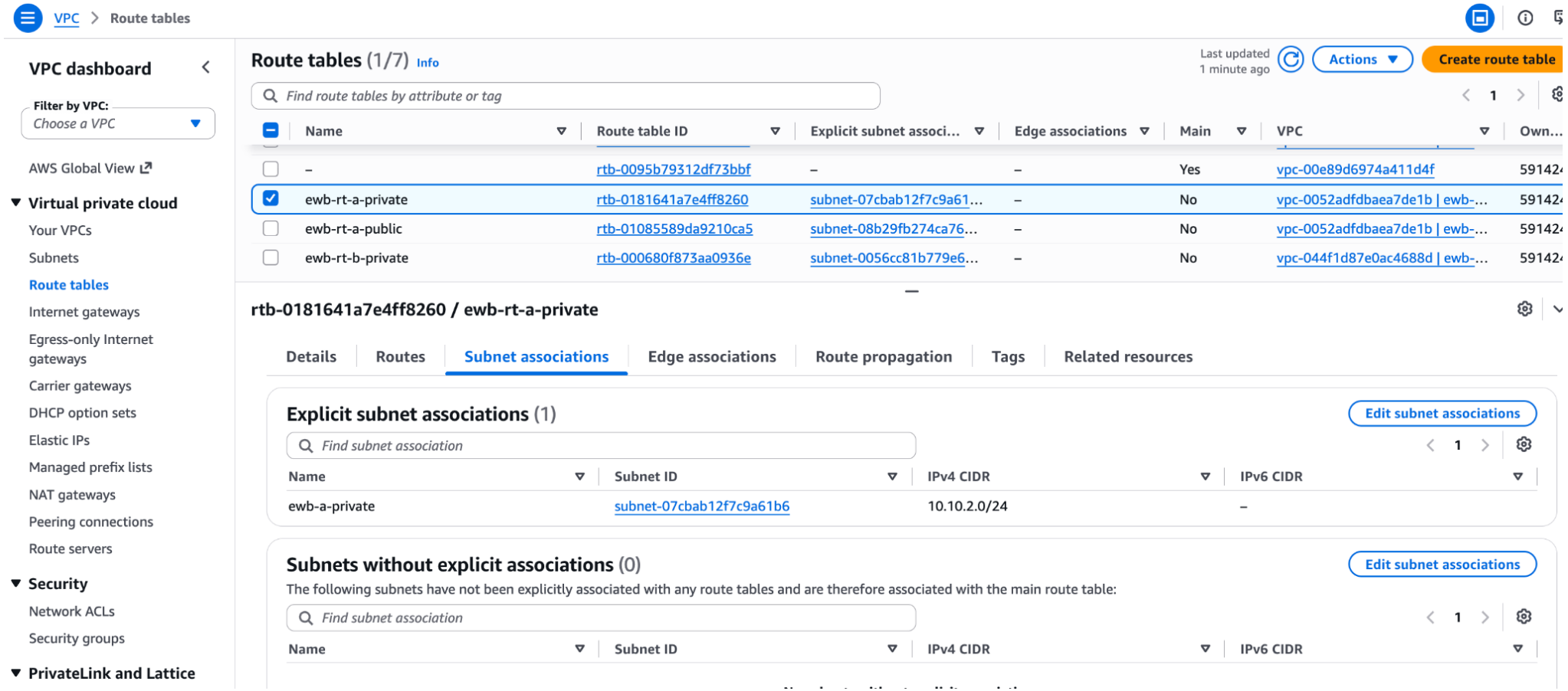
VPC A private routes

Local 10.10.0.0/16 route plus 10.20.0.0/16 through the peering connection.



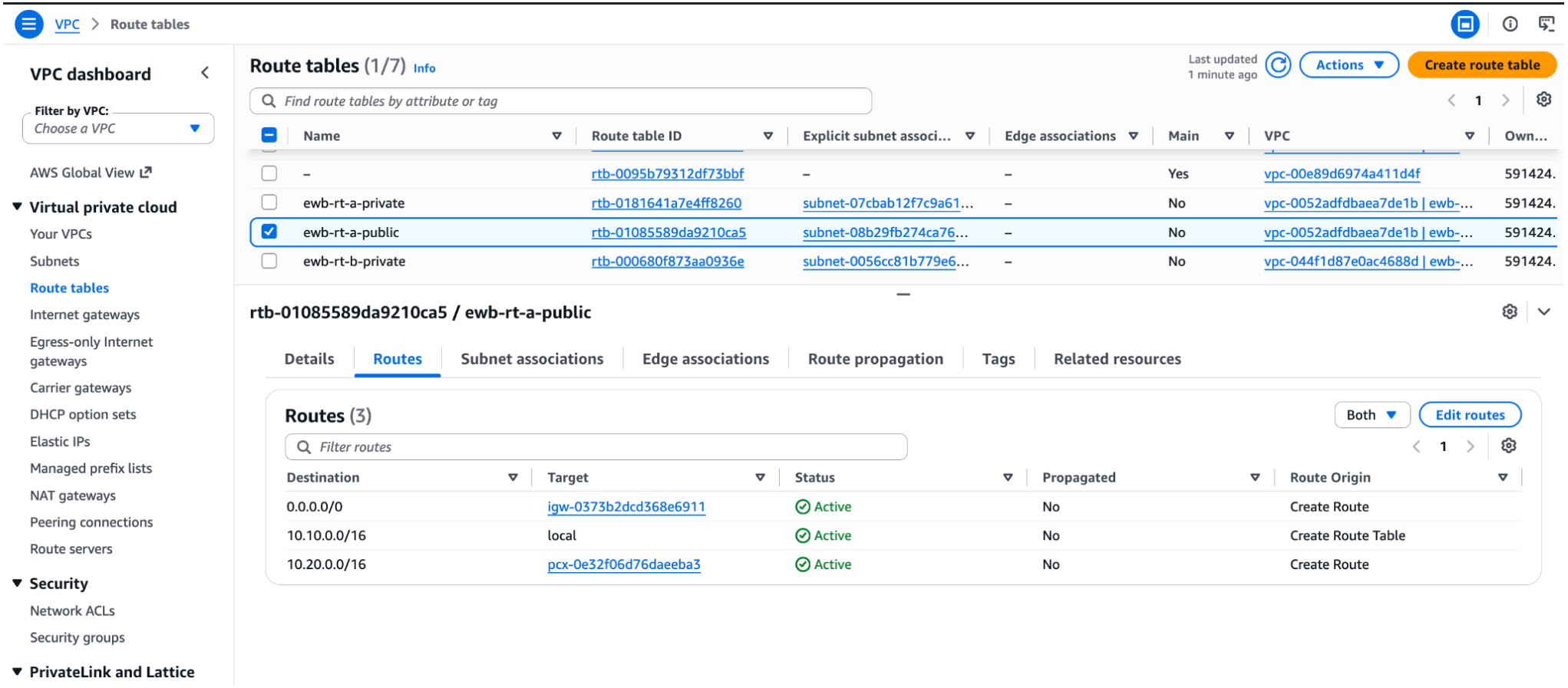
VPC A private subnet association

ewb-a-private, 10.10.2.0/24, is explicitly associated with ewb-rt-a-private.



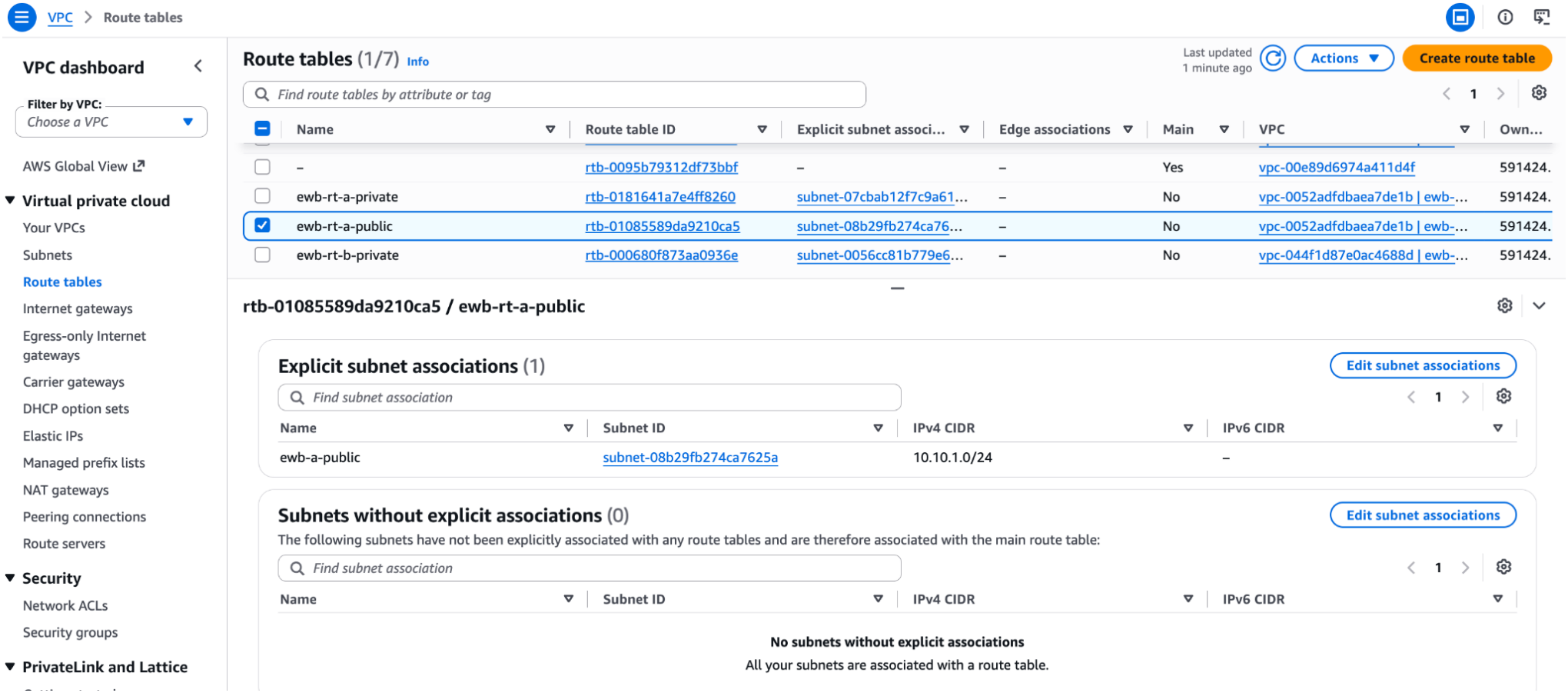
VPC A public routes

Local route, internet default route and the VPC B peer route needed by the bastion.



VPC A public subnet association

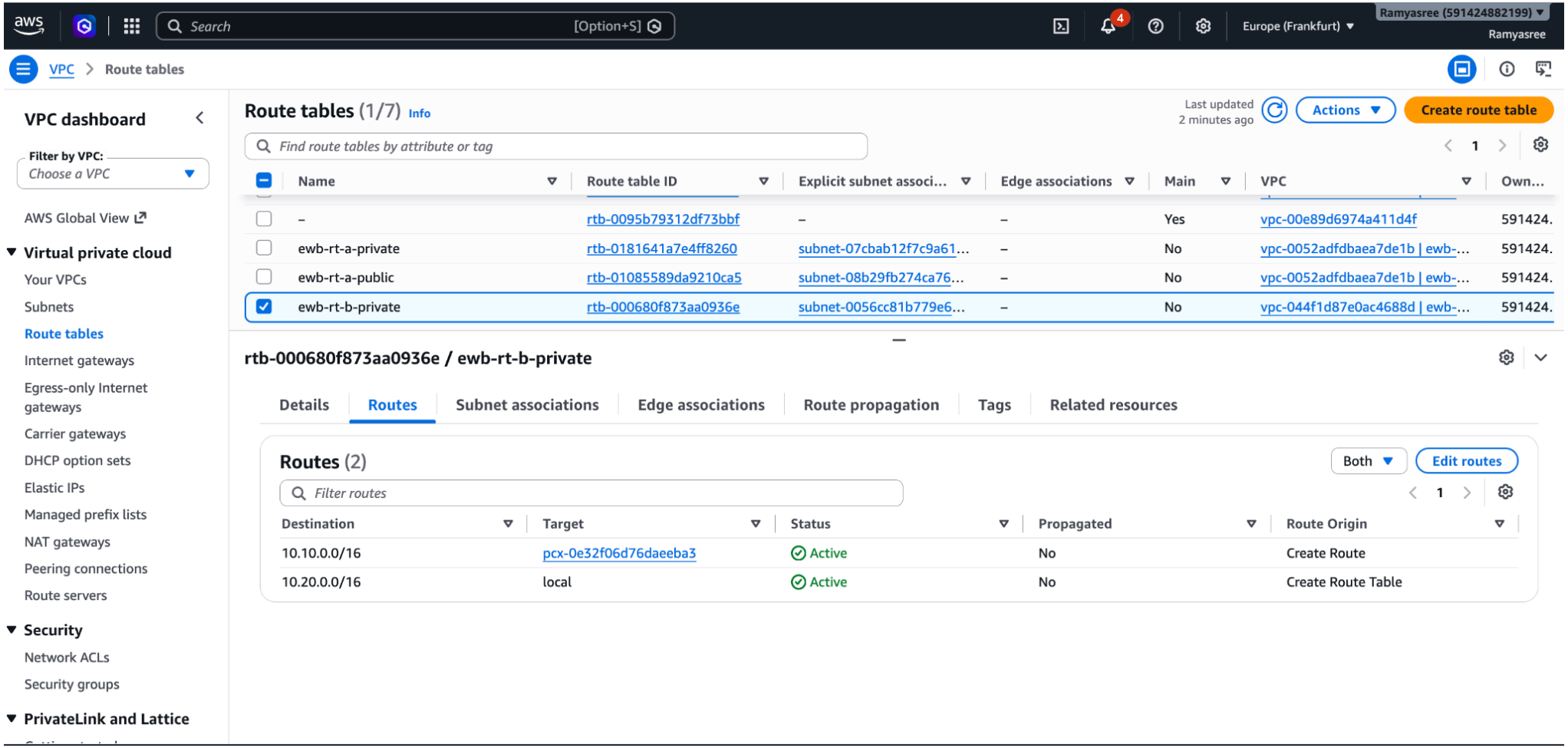
ewb-a-public, 10.10.1.0/24, is explicitly associated with ewb-rt-a-public.



EVIDENCE / E11

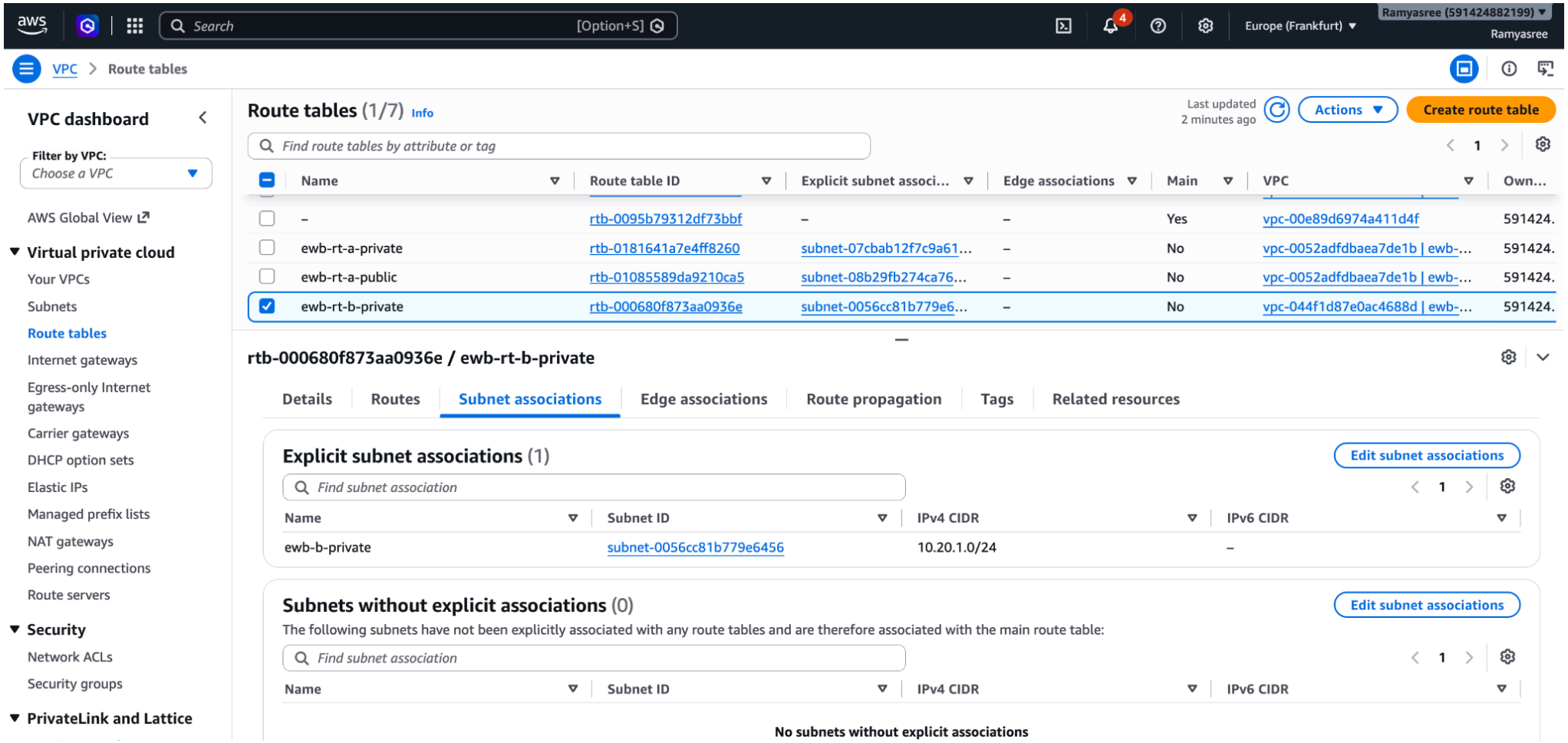
VPC B private routes

Local 10.20.0.0/16 route and return route to 10.10.0.0/16 through peering.



VPC B private subnet association

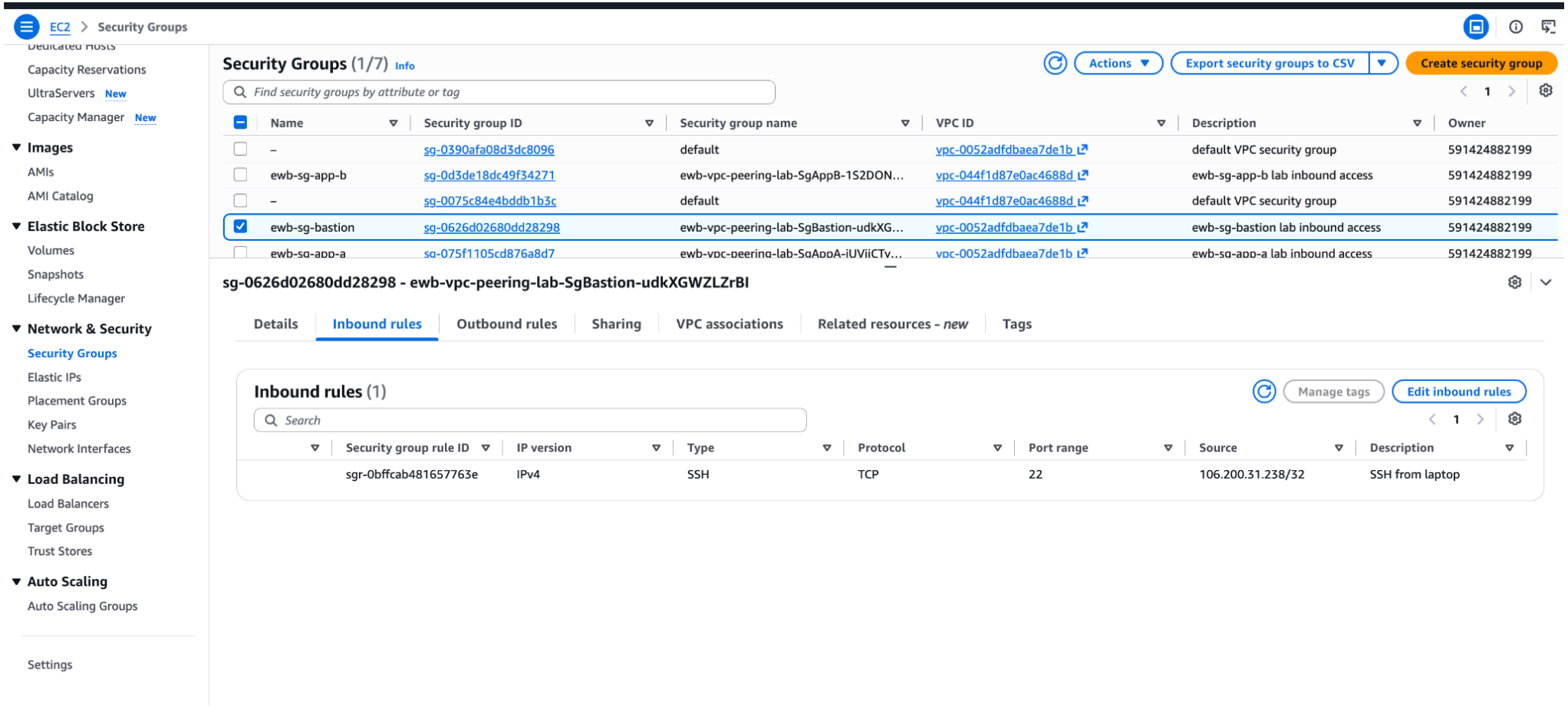
ewb-b-private, 10.20.1.0/24, is explicitly associated with ewb-rt-b-private.



EVIDENCE / E13a

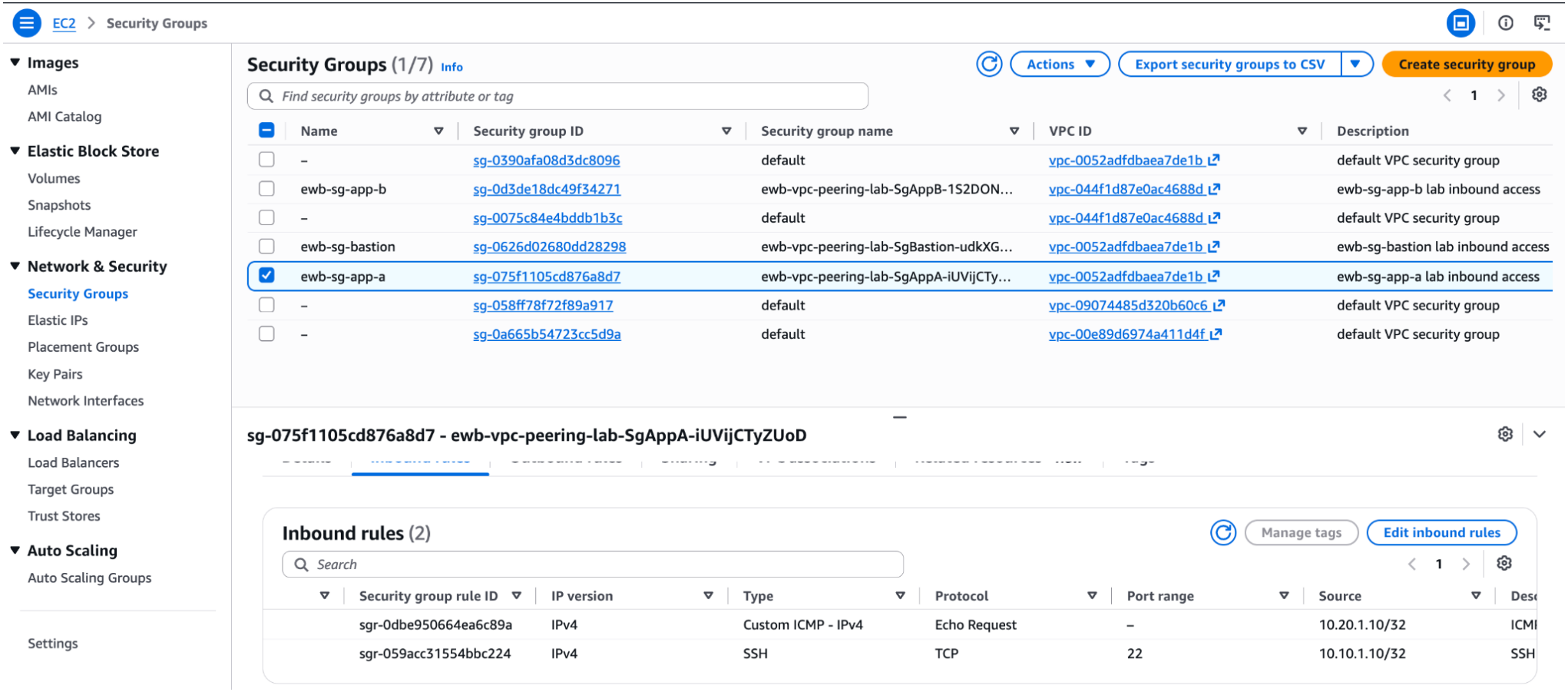
Bastion inbound SSH rule

Only TCP 22 from the captured laptop public IPv4 /32 is allowed.



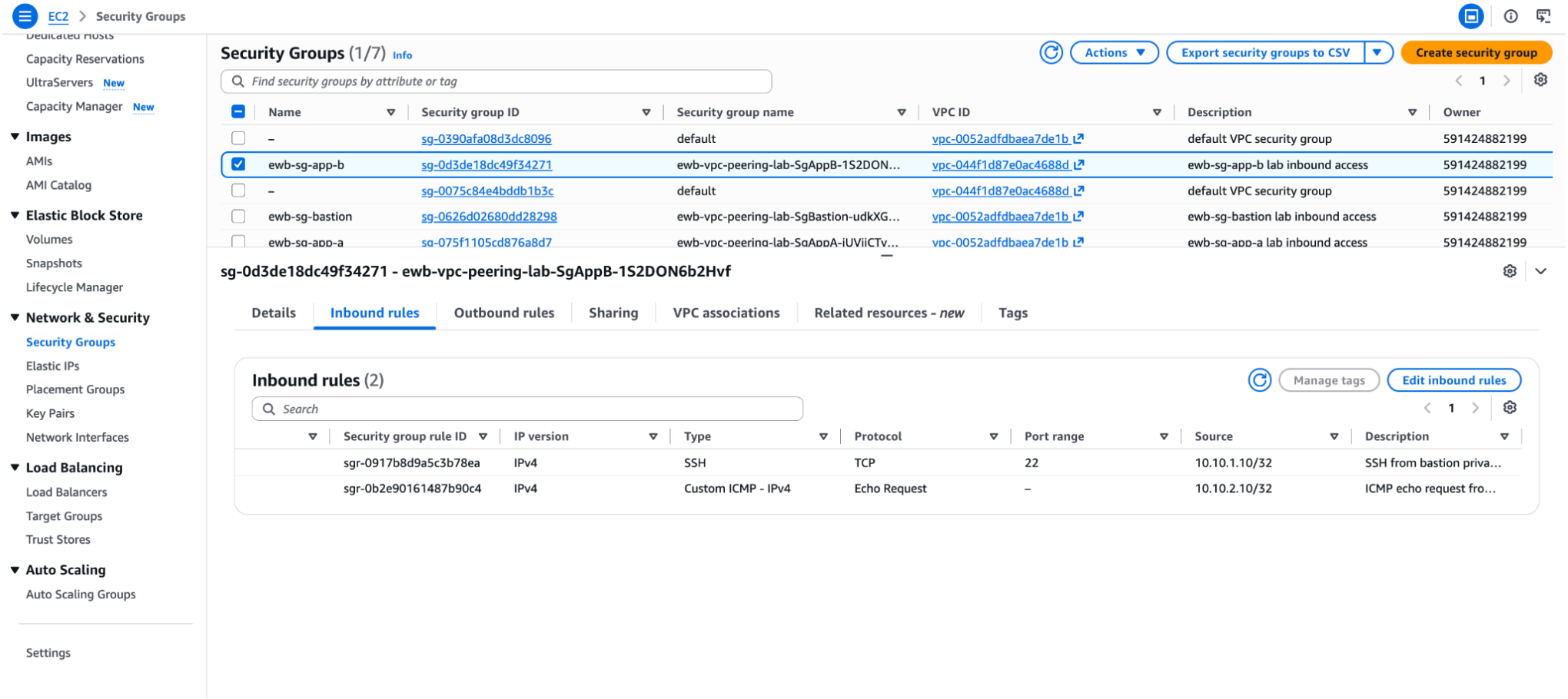
App-A inbound rules

SSH from bastion 10.10.1.10/32; Echo Request from App-B 10.20.1.10/32.



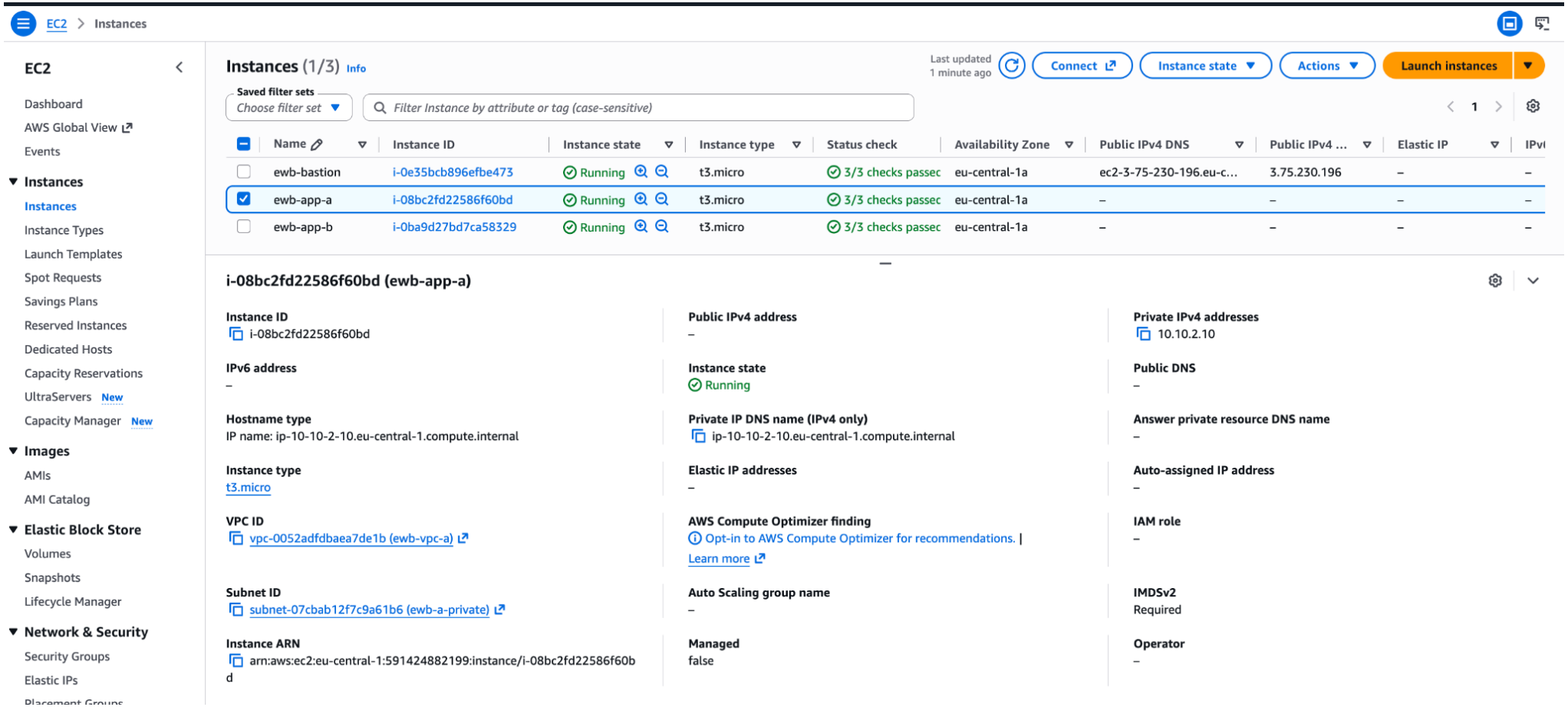
App-B inbound rules after recovery

SSH from bastion 10.10.1.10/32; Echo Request from App-A 10.10.2.10/32.



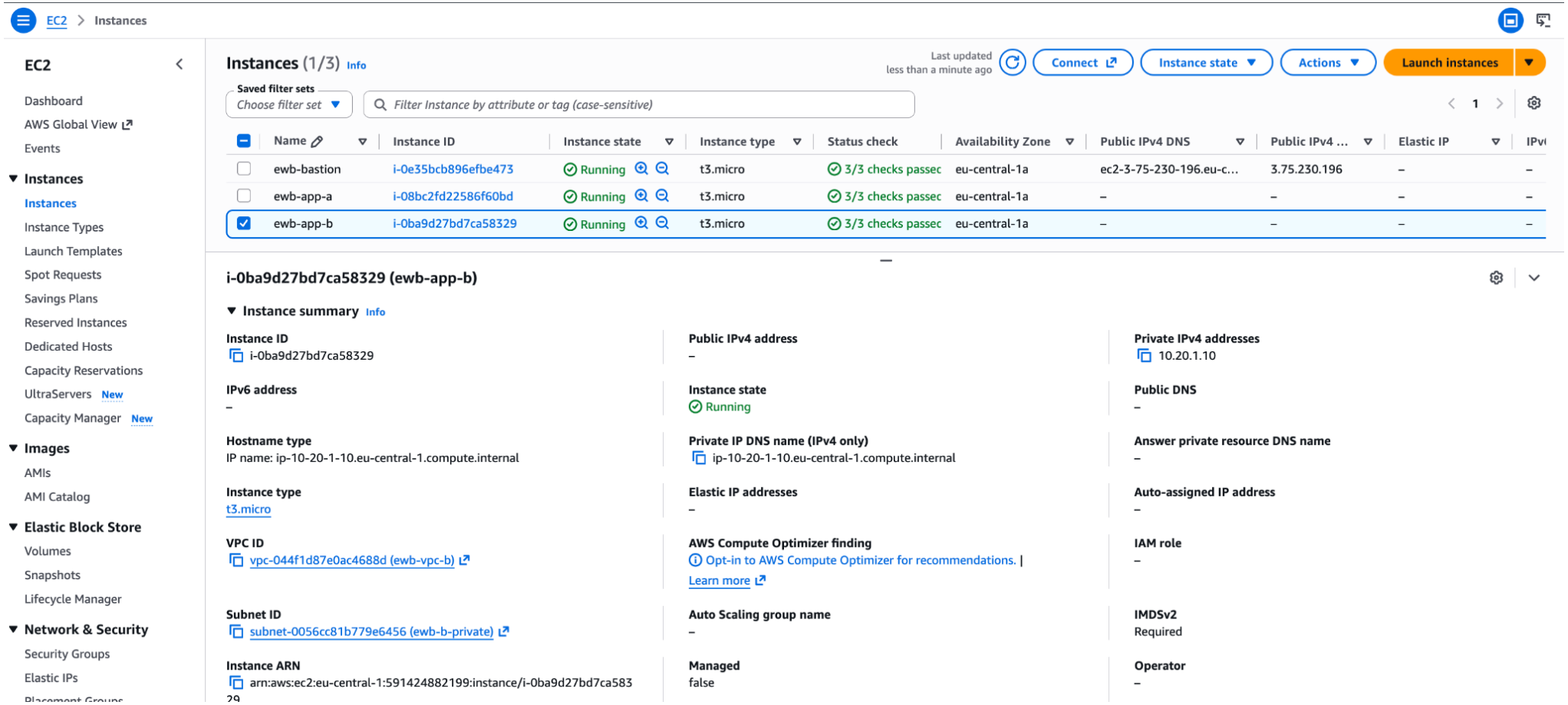
Bastion and App-A instance inventory

App-A has private IPv4 10.10.2.10 and no public IPv4. The list also shows the bastion public address and all three running instances.



App-B is private

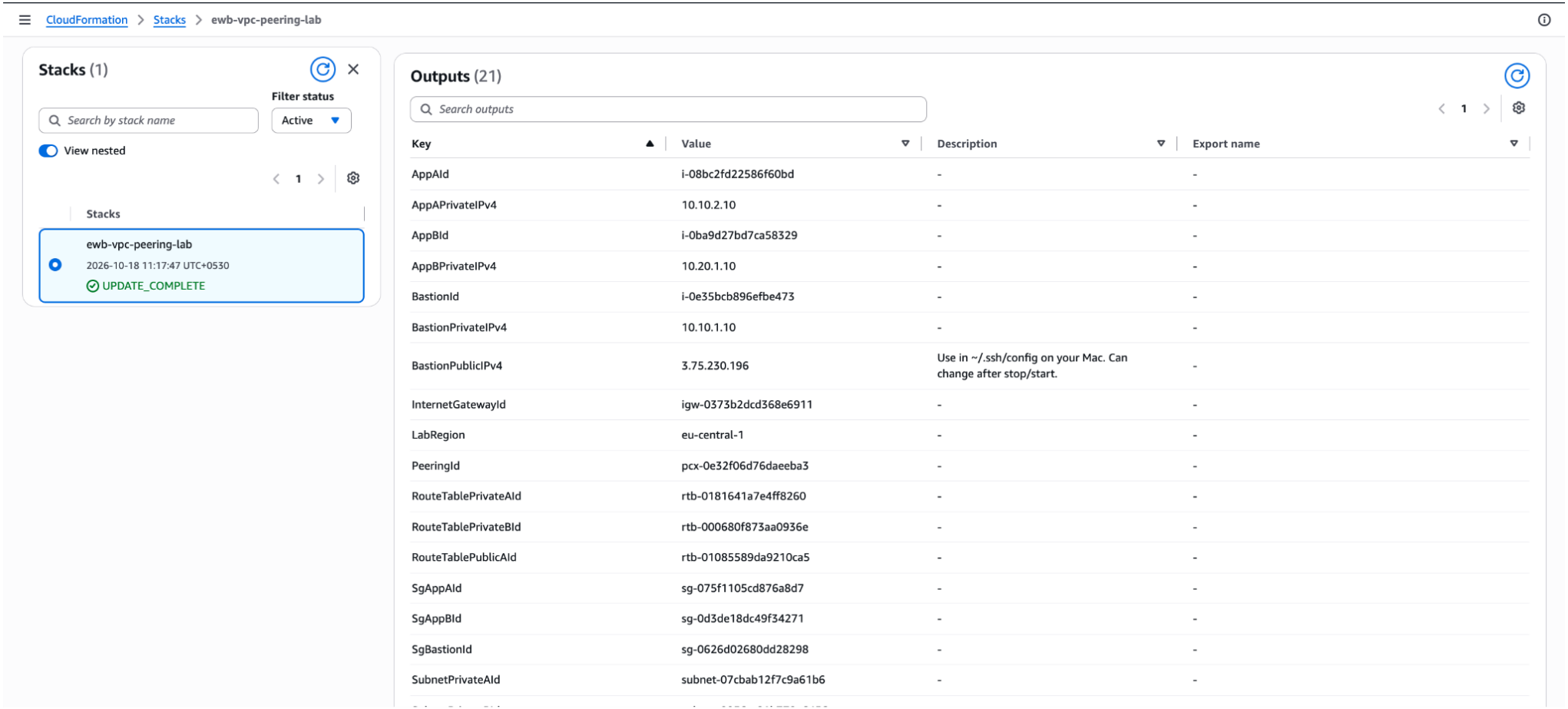
App-B i-0ba9d27bd7ca58329 has private IPv4 10.20.1.10 and no public IPv4, in the VPC B private subnet.



EVIDENCE / E16a

CloudFormation Outputs: compute

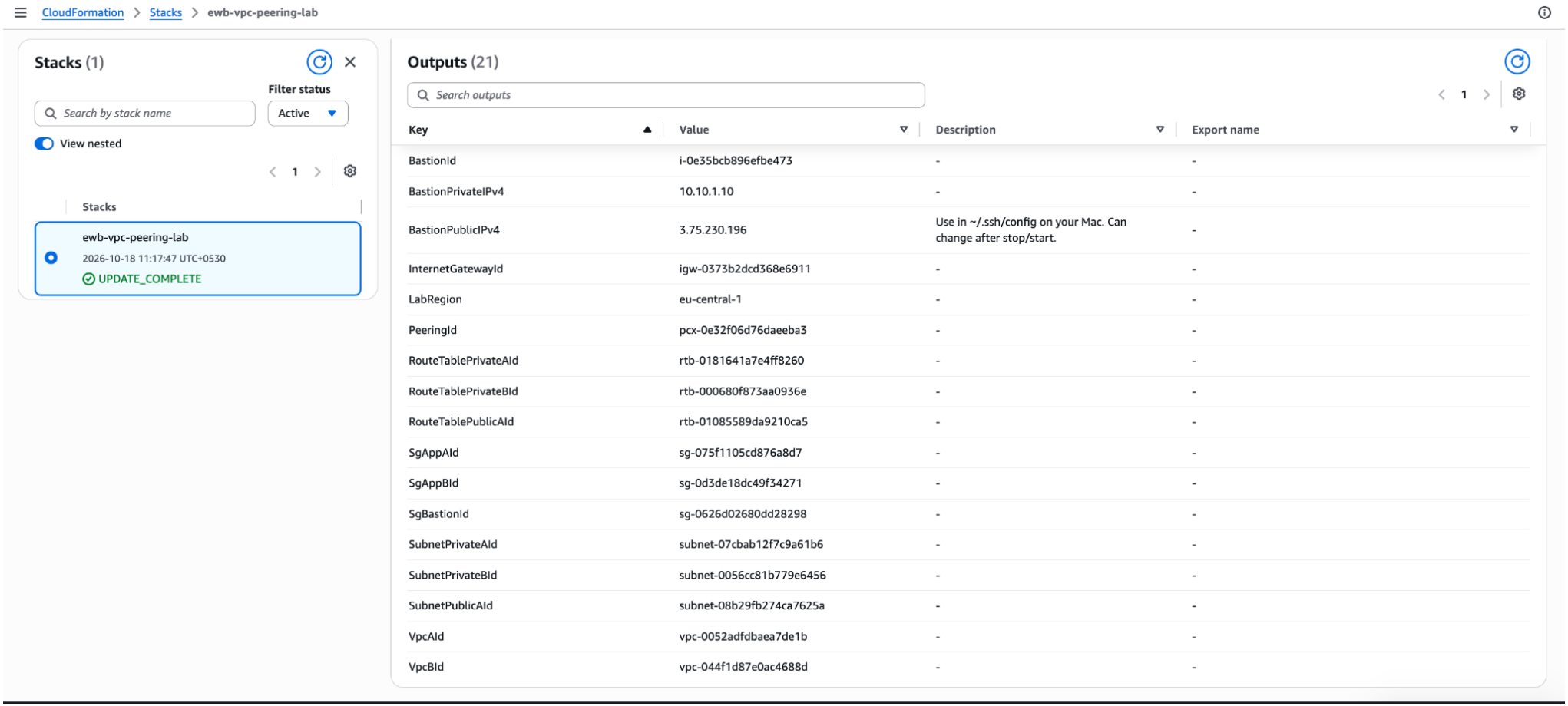
Actual compute IDs, private IPs, bastion public IP and network resource references. Stack status is UPDATE_COMPLETE.



EVIDENCE / E16b

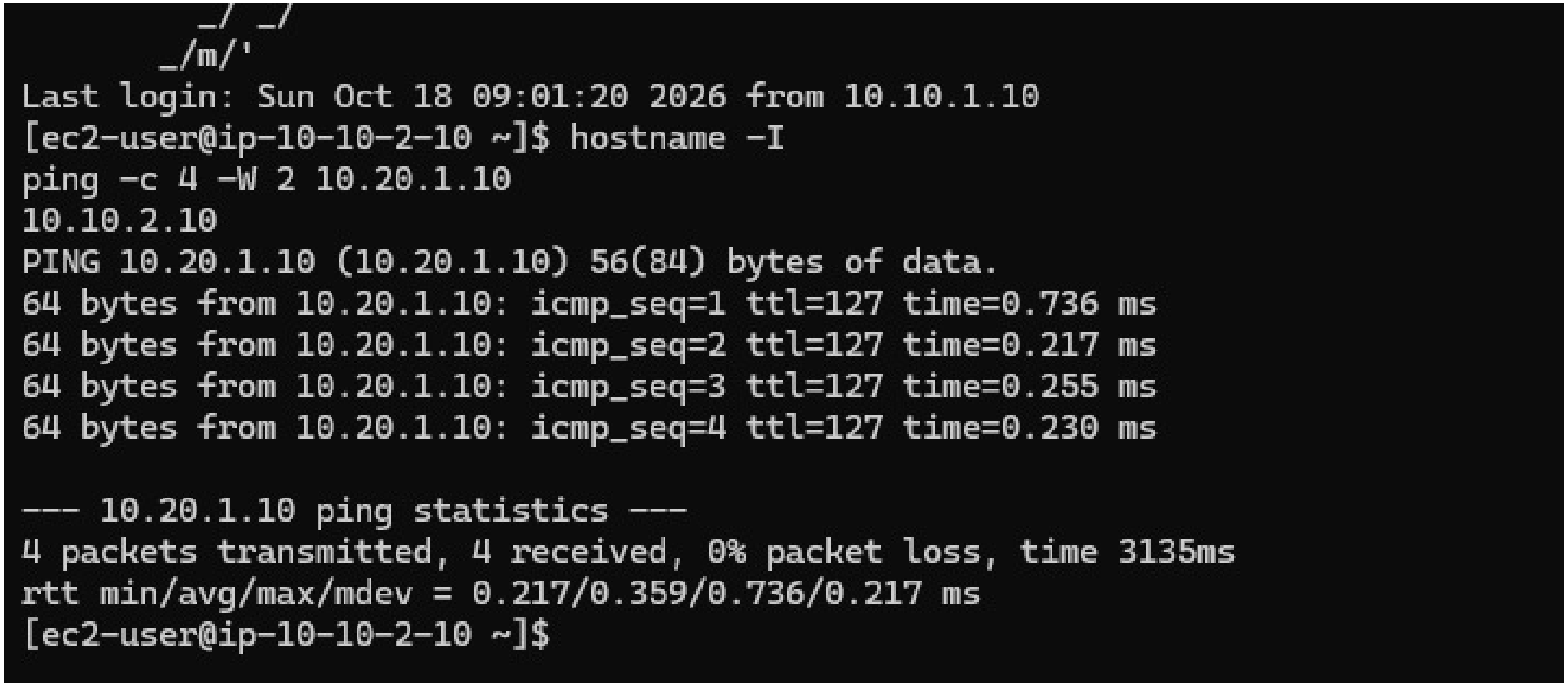
CloudFormation Outputs: networking

Actual VPC, subnet, route table, security group, gateway and peering IDs used in the inventory.



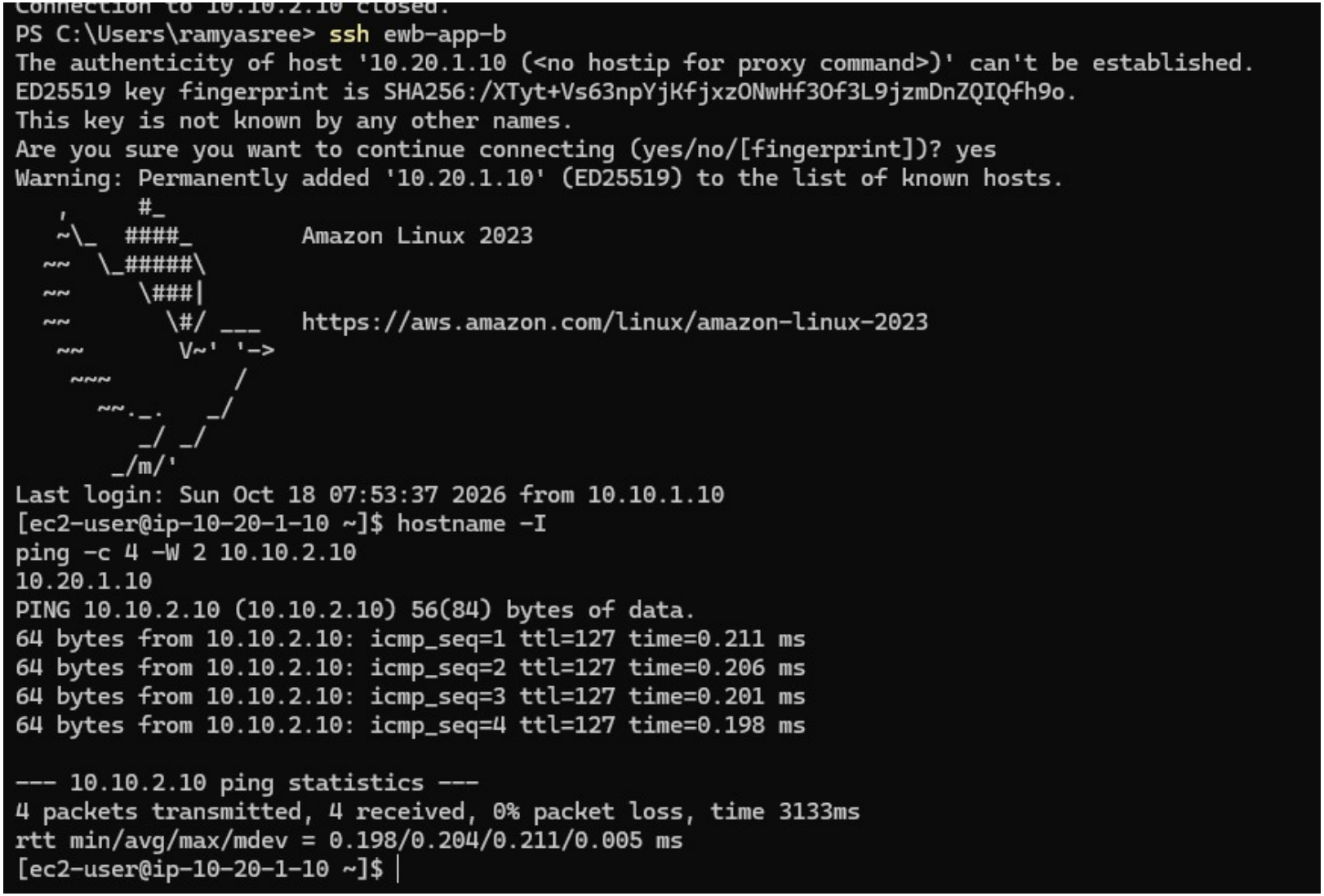
Windows: App-A to App-B

App-A 10.10.2.10 reaches App-B 10.20.1.10: four replies, 0% packet loss. Windows retest on 19 October 2026.



Windows: App-B SSH and reverse ping

PowerShell launches ssh ewb-app-b; App-B shell confirms 10.20.1.10 and receives four replies from App-A. Windows retest on 19 October 2026.



Sources and evidence notes

Assignment: EWB-Courses-AWS-VPC-Peering-Student-Project(1).pdf, 14-page workbook supplied by the student. The workbook defines the design, validation and final cleanup requirements.

Execution sources: Original student-uploaded terminal and AWS Console screenshots reproduced in E01-E16b and W01-W02. Evidence is embedded without changing reported commands or results. Selected screenshots show troubleshooting history; captions identify the valid test being used.

Method: CloudFormation provisioning is explicitly disclosed. This report documents the working design and observed tests; it does not imply every resource was created manually through the Console.

Official AWS references

VPC peering resource and same-account acceptance
https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-resource-ec2-vpcpeeringconnection.html

VPC peering concepts
https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html

EC2 stop and start
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/Stop_Start.html

Peering routes
https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-routing.html

Security group behavior
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html

EC2 pricing
https://aws.amazon.com/ec2/pricing/on-demand/

Final completion boundary

Network build and validation evidence are complete. Retention is planned; stopped-state evidence, cleanup completion and cleanup screenshots remain outstanding. Windows forward/reverse repeat-test evidence has been added. No credentials or private-key material are included.

Resource termination evidence

EWB VPC Peering | Frankfurt (eu-central-1) | Cleanup evidence addendum

EC2 termination: All three project instances are shown as **Terminated** in the supplied EC2 screenshot. This confirms compute termination in the Frankfurt Region; it does not by itself confirm removal of every related resource.

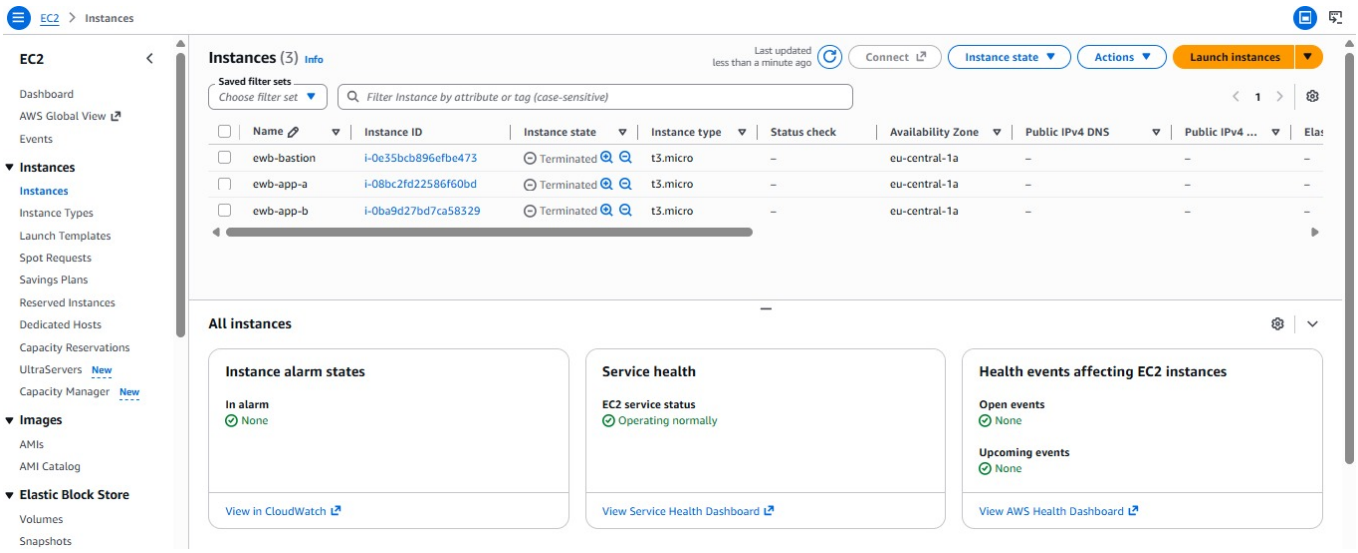


Figure C1. EC2 console - three instances terminated

ewb-bastion | i-0e35bcb896efbe473 | Terminated

ewb-app-a | i-08bc2fd22586f60bd | Terminated

ewb-app-b | i-0ba9d27bd7ca58329 | Terminated

Interpretation: The screenshots document instance termination as part of lab cleanup. A final inventory check of remaining EBS volumes, public IP allocations, networking resources, and billing is still recommended.

CloudFormation deletion evidence

EWB VPC Peering | Frankfurt (eu-central-1) | Cleanup evidence addendum

Stack: ewb-vpc-peering-lab. Two screenshots show the deletion request being initiated and the stack in **DELETE_IN_PROGRESS**. The empty Active stacks view does **not** establish **DELETE_COMPLETE**.

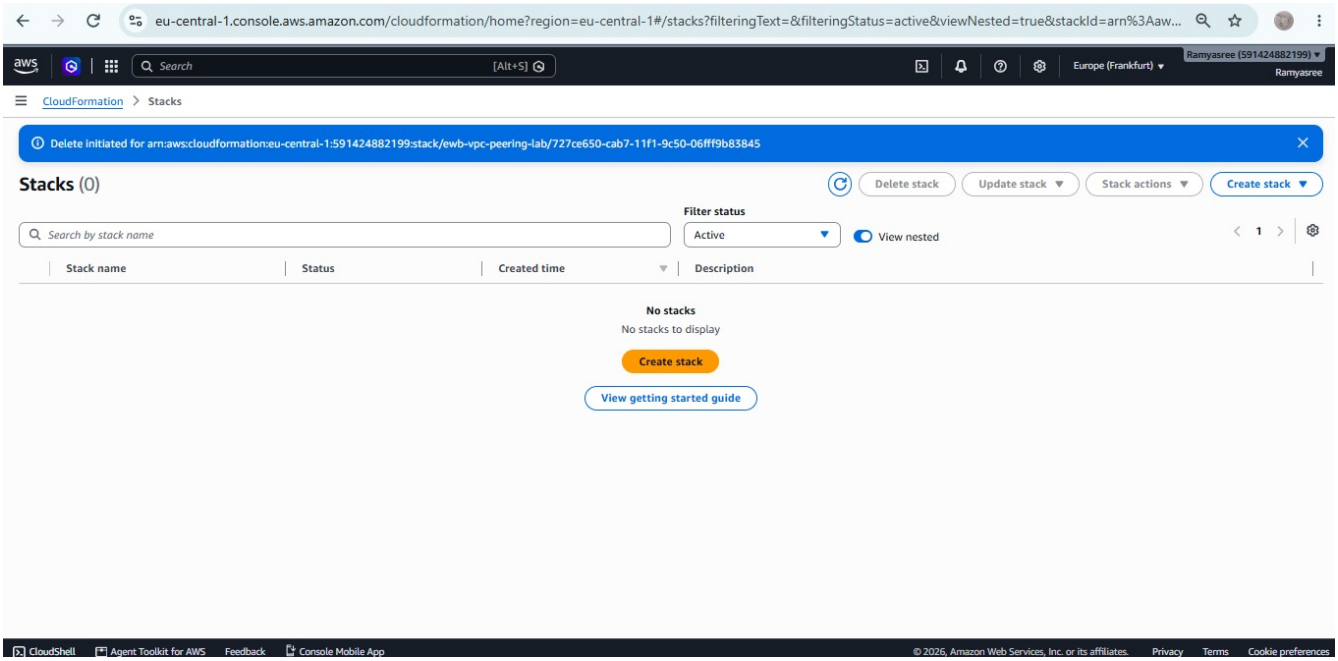


Figure C2. Deletion request initiated - Active list empty

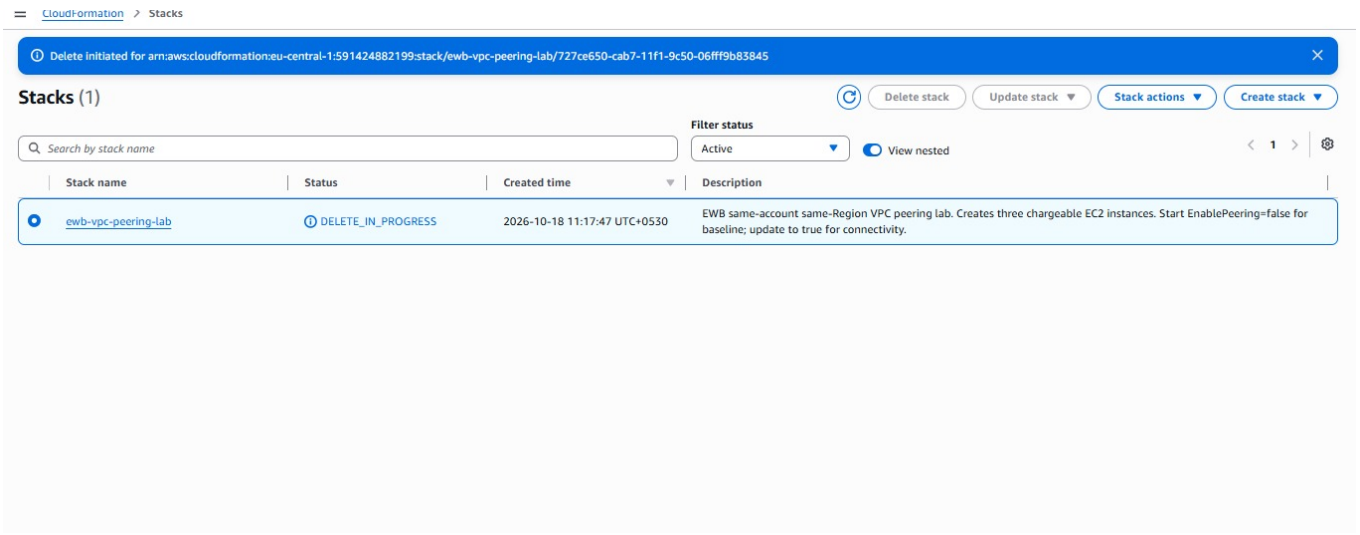


Figure C3. CloudFormation stack - DELETE_IN_PROGRESS

Final verification required: Set CloudFormation stack filter to Deleted and confirm **DELETE_COMPLETE** in stack history; inspect VPC, EC2/EBS and Elastic IP inventory for any retained resources. This addendum updates the earlier cleanup status, but does not claim final deletion confirmation.